

Hacker Holidays - Day 4 - Packet light

#NetworkForensics

#PCAPAnalysis

#Cryptography

No.	Time	Source	Destination	Protocol	Length	Cookie	Info
1	0.000000	127.0.0.1	127.0.0.1	TCP	56		53525 → 28194 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
2	0.000019	127.0.0.1	127.0.0.1	TCP	44		28194 → 53525 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
3	0.000023	127.0.0.1	127.0.0.1	TCP	56		[TCP Port numbers reused] 53525 → 28194 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
4	0.000057	127.0.0.1	127.0.0.1	TCP	44		28194 → 53525 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
5	1.244410	BelkinIntern.cc:a7:...	ASUSTekCOMPU.df:b2:...	ARP	60		Who has 192.168.1.141? Tell 192.168.1.1
6	1.244429	ASUSTekCOMPU.df:b2:...	BelkinIntern.cc:a7:...	ARP	42		192.168.1.141 is at e8:9c:25:df:b2:51
7	1.000355	127.0.0.1	127.0.0.1	TCP	56		53525 → 28194 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
8	1.500366	127.0.0.1	127.0.0.1	TCP	44		28194 → 53525 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
9	2.000004	127.0.0.1	127.0.0.1	TCP	56		[TCP Port numbers reused] 53525 → 28194 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
10	2.000027	127.0.0.1	127.0.0.1	TCP	44		28194 → 53525 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
11	2.515546	127.0.0.1	127.0.0.1	TCP	56		[TCP Port numbers reused] 53525 → 28194 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
12	2.515566	127.0.0.1	127.0.0.1	TCP	44		28194 → 53525 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
13	2.570046	192.168.1.141	34.41.103.191	TCP	66		50419 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
14	2.601748	34.41.103.191	192.168.1.141	TCP	66		8080 → 50419 [SYN, ACK] Seq=0 Ack=1 Win=65280 Len=0
15	2.601830	192.168.1.141	34.41.103.191	TCP	54		50419 → 8080 [ACK] Seq=1 Ack=1 Win=65280 Len=0
16	2.602014	192.168.1.141	34.41.103.191	HTTP	482		GET /temp/updates.py HTTP/1.1
17	2.639253	34.41.103.191	192.168.1.141	TCP	60		8080 → 50419 [ACK] Seq=1 Ack=429 Win=64896 Len=0
18	2.644495	34.41.103.191	192.168.1.141	TCP	245		8080 → 50419 [PSH, ACK] Seq=1 Ack=429 Win=64896 Len=191 [TCP PDU reassembled in 19]
19	2.644903	34.41.103.191	192.168.1.141	HTTP	1140		HTTP/1.0 200 OK (text/x-python)
20	2.644947	192.168.1.141	34.41.103.191	TCP	54		50419 → 8080 [ACK] Seq=429 Ack=1279 Win=64256 Len=0
21	2.645192	192.168.1.141	34.41.103.191	TCP	54		8080 → 50419 [FIN, ACK] Seq=429 Ack=1279 Win=64256 Len=0
22	2.645413	192.178.192.138	192.168.1.141	TLVSV1.2	218		Application Data
23	2.645733	192.168.1.141	192.178.192.138	TLVSV1.2	92		Application Data
24	2.662166	192.178.192.138	192.168.1.141	TCP	60		443 → 50637 [ACK] Seq=165 Ack=39 Win=992 Len=0
25	2.665228	34.41.103.191	192.168.1.141	TCP	60		8080 → 50419 [ACK] Seq=1279 Ack=430 Win=64896 Len=0
26	3.020052	127.0.0.1	127.0.0.1	TCP	56		[TCP Port numbers reused] 53525 → 28194 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM

Frame 1: Packet, 56 bytes on wire (448 bits), 56 bytes captured (448 bits) on interface \Device\NPF_{...} Loopback, Null/Loopback

Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

Transmission Control Protocol, Src Port: 53525, Dst Port: 28194, Seq: 0, Len: 0

traffic.pcapng

Packets: 1348

Profile: Default

I filtered the capture for traffic using TCP port 8080 :

```
tcp.port == 8080
```

The results showed repeated communication between 192.168.1.141 and 34.41.103.191 .

No.	Time	Source	Destination	Protocol	Length	Info
520	16.459520	192.168.1.141	34.41.103.191	HTTP	280	GET / HTTP/1.1
519	16.459443	192.168.1.141	34.41.103.191	TCP	54	59531 → 8080 [ACK] Seq=1 Ack=1 Win=65280 Len=0
505	16.426750	192.168.1.141	34.41.103.191	TCP	66	59531 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
440	16.126300	192.168.1.141	34.41.103.191	TCP	54	57725 → 8080 [FIN, ACK] Seq=227 Ack=5323 Win=64512 Len=0
430	16.126281	192.168.1.141	34.41.103.191	TCP	54	57725 → 8080 [ACK] Seq=227 Ack=5323 Win=64512 Len=0
437	16.125651	192.168.1.141	34.41.103.191	TCP	54	57725 → 8080 [ACK] Seq=227 Ack=4448 Win=65280 Len=0
424	16.125387	192.168.1.141	34.41.103.191	TCP	54	57725 → 8080 [ACK] Seq=227 Ack=1608 Win=65280 Len=0
438	16.078299	192.168.1.141	34.41.103.191	HTTP	280	GET / HTTP/1.1
427	16.078212	192.168.1.141	34.41.103.191	TCP	54	57725 → 8080 [ACK] Seq=1 Ack=1 Win=65280 Len=0
424	16.038022	192.168.1.141	34.41.103.191	TCP	66	57725 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
418	15.993907	192.168.1.141	34.41.103.191	TCP	54	50436 → 8080 [FIN, ACK] Seq=227 Ack=5323 Win=64512 Len=0
417	15.993809	192.168.1.141	34.41.103.191	TCP	54	50436 → 8080 [ACK] Seq=227 Ack=5323 Win=64512 Len=0
414	15.993395	192.168.1.141	34.41.103.191	TCP	54	50436 → 8080 [ACK] Seq=227 Ack=4448 Win=65280 Len=0
411	15.993146	192.168.1.141	34.41.103.191	TCP	54	50436 → 8080 [ACK] Seq=227 Ack=1608 Win=65280 Len=0
391	15.954415	192.168.1.141	34.41.103.191	HTTP	280	GET / HTTP/1.1
390	15.954330	192.168.1.141	34.41.103.191	TCP	54	50436 → 8080 [ACK] Seq=1 Ack=1 Win=65280 Len=0
385	15.926213	192.168.1.141	34.41.103.191	TCP	66	50436 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
21	2.645192	192.168.1.141	34.41.103.191	TCP	54	50419 → 8080 [FIN, ACK] Seq=429 Ack=1279 Win=64256 Len=0
20	2.644947	192.168.1.141	34.41.103.191	TCP	54	50419 → 8080 [ACK] Seq=429 Ack=1279 Win=64256 Len=0
15	2.601830	192.168.1.141	34.41.103.191	TCP	54	50419 → 8080 [ACK] Seq=1 Ack=1 Win=65280 Len=0
13	2.570046	192.168.1.141	34.41.103.191	TCP	66	50419 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
1314	33.305190	34.41.103.191	192.168.1.141	TCP	60	8080 → 59564 [ACK] Seq=5323 Ack=228 Win=65152 Len=0
1310	33.331520	34.41.103.191	192.168.1.141	HTTP	928	HTTP/1.0 200 OK (text/html)
1308	33.329637	34.41.103.191	192.168.1.141	TCP	1474	8080 → 59564 [ACK] Seq=3028 Ack=227 Win=65152 Len=1420 [TCP PDU reassembled in 1310]
1305	33.325547	34.41.103.191	192.168.1.141	TCP	1474	8080 → 59564 [PSH, ACK] Seq=1608 Ack=227 Win=65152 Len=1420 [TCP PDU reassembled in 1310]

Frame 16: Packet, 482 bytes on wire (3856 bits), 482 bytes captured (3856 bits) on interface \Device\NPF_{...} Ethernet II, Src: ASUSTekCOMPU.df:b2:51 (e8:9c:25:df:b2:51), Dst: BelkinIntern.cc:a7:9e (c0:50:27:cc:a7:9e)

Internet Protocol Version 4, Src: 192.168.1.141, Dst: 34.41.103.191

Transmission Control Protocol, Src Port: 50419, Dst Port: 8080, Seq: 1, Ack: 1, Len: 428

Source Port: 50419

Destination Port: 8080

[Stream index: 5]

[Stream Packet Number: 4]

[Conversation completeness: Complete, WITH_DATA (31)]

[TCP Segment Len: 428]

Sequence Number: 1 (relative sequence number)

Sequence Number (raw): 2866613041

[Next Sequence Number: 429 (relative sequence number)]

Acknowledgment Number: 1 (relative ack number)

Acknowledgment Number (raw): 2635041250

0030 00 ff 4d e4 00 00 47 45 34 20 2f 74 65 6d 70 2f M: GET /temp/

0031 75 70 64 61 74 65 73 2e 70 79 20 48 54 54 50 2f updates.py HTTP/

0032 31 2e 31 0d 0a 48 6f 73 74 3a 20 62 79 74 65 2d 1.1..Host: byte-

0033 0c 6f 74 75 73 2d 68 6f 74 65 6c 2e 74 68 6d 3a lotus-ho tel.tha

0034 38 30 38 30 0d 0a 43 6f 6e 6e 65 63 74 69 6f 6e 8080..Co nnection

0035 3a 20 68 65 65 70 2d 61 6c 6f 76 65 0d 0a 55 70 e keep-a live-up

0036 67 72 61 64 65 2d 49 6e 73 65 63 75 72 65 2d 52 grade-In secure-R

0037 65 71 75 65 73 74 73 3a 20 31 0d 0a 55 73 65 72 equests: 1..User

0038 2d 41 67 65 6e 74 3a 20 4d 6f 7a 69 6c 6c 61 2f -Agent: Mozilla/

0039 35 2e 30 20 28 57 0d 6e 54 6f 77 73 20 4e 54 20 5.0 (Win dows NT

0040 31 30 2e 30 3b 20 57 69 6e 36 34 3b 20 78 35 33) Applew ebKit/53

0041 2e 35 30 20 28 48 54 6d 4c 2c 20 6c 69 68 7.36 (KH TM, link

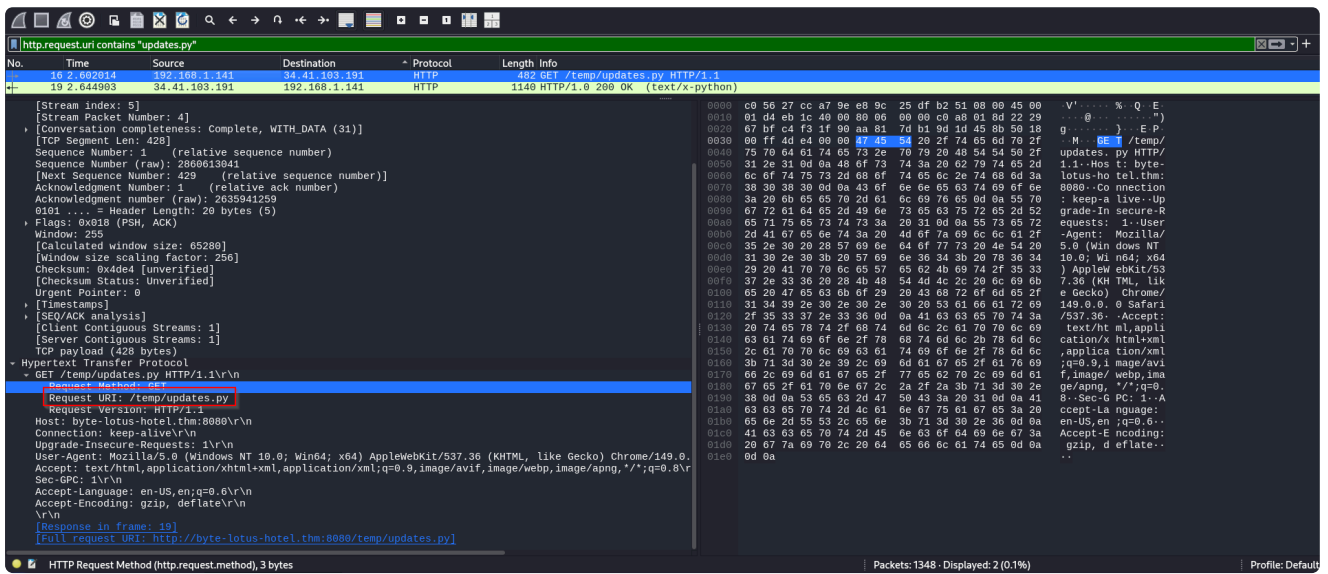
0100 65 20 47 65 63 4b 6f 29 20 43 68 72 6f 6d 65 6f e decko) Chrome/

0110 31 34 30 2e 30 3e 30 2e 30 20 53 61 66 61 72 69 149.0.0.0 Safari

I inspected the HTTP traffic and found that the client downloaded a suspicious Python script:

```
GET /temp/updates.py
```

The file came from byte-lotus-hotel.thm:8080 .



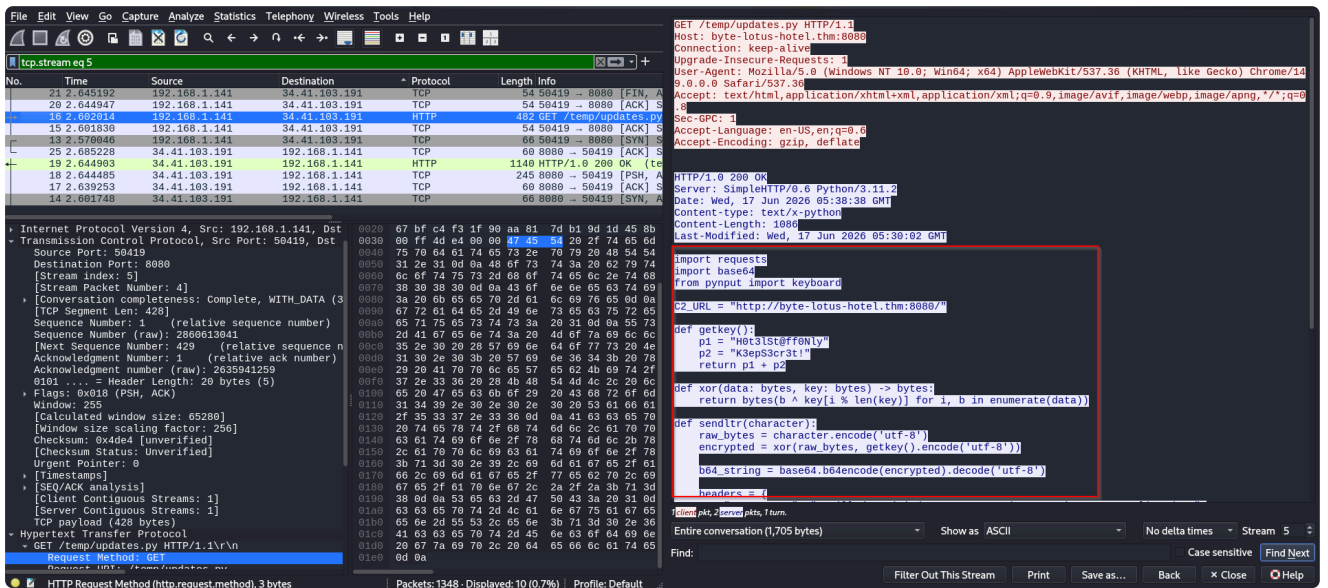
I isolated the script request with:

`http.request.uri contains "updates.py"`

This confirmed that `updates.py` was transferred shortly before the repeated HTTP requests began.

I followed the TCP stream and recovered the Python source code.

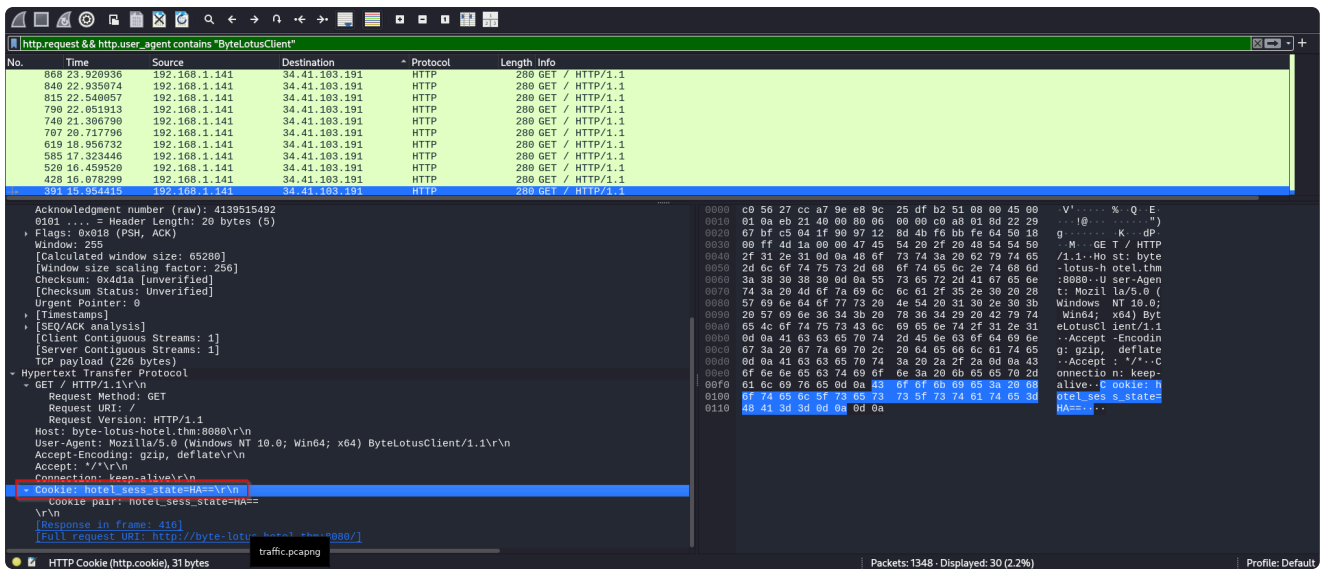
The script captured keyboard input, XOR-encrypted each character, Base64-encoded it, and placed the result inside the `hotel_sess_state` HTTP cookie.



I filtered the malicious requests using the custom User-Agent:

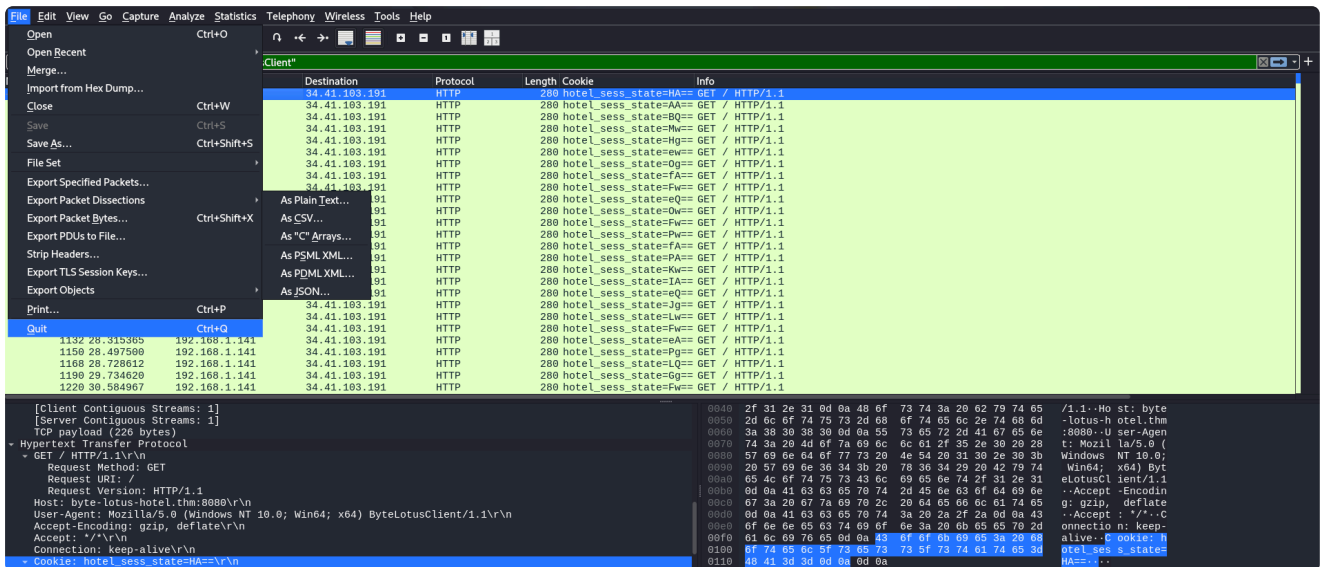
`http.request && http.user_agent contains "ByteLotusClient"`

Wireshark displayed 30 requests, each containing a different `hotel_sess_state` value.



I added the HTTP cookie as a Wireshark column and exported the 30 displayed packets as a CSV file.

This allowed me to extract the cookie values while preserving their packet order.



The CSV contained the Base64 values sent in each request. Each value represented one encrypted character.

```
Open v [icon] pack
~/thm/HackerH

config | HEAD

1 "No.", "Time", "Source", "Destination", "Protocol", "Length", "Cookie", "Info"
2 "391", "15.954415", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=HA=", "GET / HTTP/1.1 "
3 "428", "16.078299", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=AA=", "GET / HTTP/1.1 "
4 "520", "16.459520", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=BQ=", "GET / HTTP/1.1 "
5 "585", "17.323446", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=Mw=", "GET / HTTP/1.1 "
6 "619", "18.956732", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=Hg=", "GET / HTTP/1.1 "
7 "707", "20.717796", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=ew=", "GET / HTTP/1.1 "
8 "740", "21.306790", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=Og=", "GET / HTTP/1.1 "
9 "790", "22.051913", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=fA=", "GET / HTTP/1.1 "
10 "815", "22.540057", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=Fw=", "GET / HTTP/1.1 "
11 "840", "22.935074", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=eQ=", "GET / HTTP/1.1 "
12 "868", "23.920936", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=Ow=", "GET / HTTP/1.1 "
13 "907", "25.229510", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=Fw=", "GET / HTTP/1.1 "
14 "932", "25.513116", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=Pw=", "GET / HTTP/1.1 "
15 "961", "25.832942", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=fA=", "GET / HTTP/1.1 "
16 "990", "26.561195", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=PA=", "GET / HTTP/1.1 "
17 "1019", "26.820438", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=Kw=", "GET / HTTP/1.1 "
18 "1038", "26.951355", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=IA=", "GET / HTTP/1.1 "
19 "1056", "27.235316", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=eQ=", "GET / HTTP/1.1 "
20 "1076", "27.472804", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=Jg=", "GET / HTTP/1.1 "
21 "1094", "27.601632", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=Lw=", "GET / HTTP/1.1 "
22 "1114", "28.056581", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=Fw=", "GET / HTTP/1.1 "
23 "1132", "28.315365", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=eA=", "GET / HTTP/1.1 "
24 "1150", "28.497500", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=Pg=", "GET / HTTP/1.1 "
25 "1168", "28.728612", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=LQ=", "GET / HTTP/1.1 "
26 "1190", "29.734620", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=Gg=", "GET / HTTP/1.1 "
27 "1220", "30.584967", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=Fw=", "GET / HTTP/1.1 "
28 "1240", "31.645045", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=MQ=", "GET / HTTP/1.1 "
29 "1260", "31.906300", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=eA=", "GET / HTTP/1.1 "
30 "1278", "32.272819", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=PQ=", "GET / HTTP/1.1 "
31 "1300", "33.291798", "192.168.1.141", "34.41.103.191", "HTTP", "280", "hotel_sess_state=NQ=", "GET / HTTP/1.1 "
```

I created a Python script that Base64-decoded every value separately and XORed the resulting byte with the first character of the key:

```
H = 0x48
```

The values had to be decoded individually because the malware restarted the XOR operation for every captured character.

```
File Edit View Bookmarks Plugins Settings Help
GNU nano 9.0
import base64

values = ""
HA==
AA==
BQ==
Mw==
Hg==
ew==
Og==
fA==
Fw==
eQ==
Ow==
Fw==
Pw==
fA==
PA==
Kw==
IA==
eQ==
Jg==
Lw==
Fw==
eA==
Pg==
LQ==
Gg==
Fw==
MQ==
eA==
PQ==
NQ==
"".split()

key_byte = ord("H")

plaintext = "".join(
```

I executed the decoder with:

```
python3 python.py
```

The recovered flag was:

```
THM{V3r4_1s_w4tch1ng_0veR_y0u}
```



```
File Edit View Bookmarks Plugins Settings Help

(fc0d3x@fc0d3x)-[~/thm/HackerHoliday26/Forensics]
$ ls
packed-light-forensics-1784224937659.zip  packets.csv  python.py  traffic.pcapng

(fc0d3x@fc0d3x)-[~/thm/HackerHoliday26/Forensics]
$ python3 python.py
THM{V3r4_1s_w4tch1ng_0veR_y0u}
```

The covert channel used repeated HTTP requests to port 8080. The exfiltrated keystrokes were XOR-encrypted, Base64-encoded, and hidden inside the `hotel_sess_state` cookie.